Comparative Ablation of Different SSM Architectures in DreamerV3

Dimitriye Danilović Shirisha Gujja Rohan Ohlan
Nikil Thalapaneni

16 March 2026

Abstract

Since its introduction, DreamerV3[1] remains the foundational architecture for task-agnostic model-based reinforcement learning. Its RSSM component however is based on a GRU, which has known problems with long-range credit assignment and sequential training. In the time since, there has been much research into the use of different sequence model backbones, such as transformers or SSSMs. Most such attempts have not performed broad comparison across different task domains, with substantial variance in implementation other than the backbone. We present a comparative analysis of various backbones on identical tasks under task-agnostic configurations mirroring the evaluation methodology in the original paper by Hafner et al.

1 Introduction

Reinforcement learning has emerged as a powerful framework for training agents to perform complex tasks through interaction with their environment.

- **Model-free RL:** Methods that learn policies directly from experience without constructing an explicit model of the environment.
- **Model-based RL:** Methods that incorporate a learned model of the environment dynamics to plan and make decisions.

Prior to DreamerV3, the standard algorithm[1] for task-agnostic use was PPO[2], a model-free algorithm. However, PPO's applicability in task-agnostic contexts was limited, with many tasks demonstrating far higher performance from purpose-tuned expert models[1]. DreamerV3 introduced a model-based architecture with substantial gains in across-the-board performance with fixed hyperparameters, far exceeding that of PPO and on many tasks that of purpose-tuned expert models.

However, DreamerV3 continued to struggle with long-term extrapolation due to its architectural core[3], a gated recurrent unit (GRU) model. GRUs have

known difficulties with long-range credit assignment, as well as computational cost due to their sequential training. Since then, attempts have been made to replace the GRU as an architectural backbone in DreamerV3 with other types of sequence models:

1. Samsami et al.[3] introduce Recall2Imagine (R2I) which utilizes a Structured State-Space Model in place of the GRU-based RSSM;
2. Wang et al.[4] introduce Drama, which utilizes a Mamba-2[5] SSSM for efficient training;
3. Mattes et al.[6] introduce Hieros, which utilizes a Simplified Structured State-Space Model (S5)[7];
4. Zhang et al.[8] introduce STORM, swapping in a transformer for the GRU;
5. Burchi and Timofte[9] build on STORM with TWISTER, modifying the loss function to better accomodate the transformer swap.

However, each of these works introduces additional architectural or methodological changes alongside the backbone swap, and evaluates on different subsets of environments. No prior work provides a controlled comparison isolating the sequence model as the sole variable. Herein we present such a controlled comparison.

2 Related Work

2.1 Recall2Imagine — S4-variant backbone

Recall2Imagine[3] replaces the GRU with a multi-layer S4 variant (S3M), with parallel scan during training and sequential inference during imagination. This assists with long-range gradient flow and computational efficiency during training. However, they also make a change to the policy input, selecting from three different policy conditioning schemes based on environment type: output state policy $\pi(a \mid z_t, h_t)$, hidden state policy $\pi(a \mid z_t, x_t)$, or full state policy $\pi(a \mid z_t, h_t, x_t)$. This per-domain tuning makes comparison ambiguous as it's unclear how much of the gain comes from the backbone vs. the policy conditioning choice. Per their ablation, $\pi(a \mid z_t, h_t)$ under-performs on memory tasks, due to state compression. This is an architectural quality of SSSMs. Hence, for architectural uniformity, we elect to use $\pi(a \mid z_t, x_t)$ for all SSSM-based models studied.

2.2 Drama — Mamba-2 variant backbone

Drama[4] replaces the GRU with a Mamba-2 SSSM[5], however, it also substantially restructures the overall model. DreamerV3's core novelty is the prior/posterior categorical latent with KL balancing, which Drama drops. The autoencoder architecture is also substantially altered, using a VAE trained on

only the current observation, with no deterministic state input. Reward and termination predictors are single fully-connected layers from h_t, rather than the MLP heads from concatenated z_t, h_t used in DreamerV3. They also introduce a new sampling methodology, dynamic frequency-based sampling, an orthogonal contribution which their ablation credits with nearly a quarter of the improvement. They also omit DreamerV3's robustness techniques for reward prediction.

2.3 Hieros — S5 backbone

Hieros[6] utilizes a S5[7] backend, but their core contribution is a goal-conditioned hierarchical policy structure. They ablate the hierarchical policy structure against the S5-backbone without it, but they do not broadly compare the S5-backbone alone with DreamerV3. They encountered similar issues with policy as R2I did.

2.4 STORM — Transformer backbone

STORM[8] utilizes a GPT-style transformer[10] backend. It's generally the closest in architecture of all backbone-swaps reviewed, with no significant differences other than the backbone.

2.5 TWISTER — Transformer backbone, optimized loss function

TWISTER[9] utilizes the same transformer backend as STORM, but introduces a modified loss objective more well-suited for transformers. This is architecturally motivated, and not a typical confound, but useful to ablate in the overall comparison. It does utilize segment-level caching, which STORM does not, where hidden state sequences from previous segments are cached and reused as extended context. Notably however, they utilize relative positional encodings as introduced in Transformer-XL[11] instead of absolute positional encodings as in STORM.

3 Proposed Approach

To harmonize the ablation, we will implement a unified architecture across backbones.

3.1 SSSM Policy Structure

All three SSSM approaches will be implemented using hidden state policy $\pi(a \mid z_t, x_t)$ on all tasks. The standard DreamerV3 $\pi(a \mid z_t, h_t)$ policy was demonstrated to under-perform significantly on memory tasks due to state compression which is inherent to the SSSM architecture by Samsami et al.[3] Other policy

choices showed some gain on other tasks compared to the selected policy, however given the stated motivation of Dreamer-type models as task-agnostic with no change to hyperparameters, the choice of a single unified policy structure best suits the purposes of this analysis.

3.2 Drop-in Mamba

Notably, a drop-in replacement of the GRU with a Mamba-2 SSSM[5] will be performed, without the architectural differences posed by Drama[4].

3.3 Transformer Positional Encoding

All transformer runs will be done with Transformer-XL-style[11] relative positional encoding and segment-caching. The key difference between them will be the use of CPC or not.

4 Experiment

The core experiment will be the comparative ablation between the backbones with all else held equal, as well as the GRU DreamerV3 baseline. However, we also pose two sub-experiments:

4.1 Dynamic Frequency-based Sampling

The dynamic frequency-based sampling strategy used by Drama[4] demonstrated substantial improvement over uniform sampling of training data, which we implement for each model as a subexperiment. They maintain two per-transition count vectors:

- v — how many times a transition has been sampled for world model training,
- b — how many times a transition has been sampled for imagination / policy training.

For world model sampling, the sampling probabilities are a simple softmax $\sigma(-\mathbf{v})$. This is based on the anti-frequency sampling approach introduced by Robine et al.[12, 4]

For imagination sampling, the probabilities are given by

$$\sigma(f(v,b)), \text{where } f(v,b) = v - b - \max(0, v - b).$$

Thus, when $v_i \geq b_i$ (the world model has seen this transition at least as often as the policy model has), $f = 0$. When $v_i < b_i$ (the policy model has seen this transition more than the world model), $f = v_i - b_i$ which is negative, down-weighting the probability.

This serves to prevent training on transitions which the world-model has not yet learned to predict well.

4.2 CPC

TWISTER introduces a contrastive predictive coding loss motivated by the insight that next-state prediction does not fully exploit the capability of transformers[9]. They demonstrate a substantial gain over STORM with this methodology. Here, motivated by the Dao and Gu demonstration of the duality between transformers and SSSMs[5], we implement such CPC loss to all models for comparison against baseline. We believe there may be substantial benefit to Mamba-2 and expect limited benefit to S4/S5.

The loss is calculated as follows:

$$\mathcal{L}_{\text{CPC}} = -\frac{1}{K}\sum_{k=0}^{K-1}\log\frac{\exp(\text{sim}(z'_{t+k}, s_t))}{\sum_{z'_j \in Z'}\exp(\text{sim}(z'_j, s_t))}$$

z' denotes the encoding of augmented observation o' which is derived from o by randomly cropping and resizing. The similarity function utilized is

$$\text{sim}(z'_j, s_t) = q_\phi^k(z'_j)^T p_\phi^k(s_t, a_{t:t+k})$$

where q_ϕ^k and p_ϕ^k are two learned MLPs per each step k.

5 Expected Deliverables

5.1 Code Repository

A publicly available code repository implementing all backbone variants (GRU, S4/S3M, S5, Mamba-2, Transformer, Transformer+CPC) within a unified test harness built on the DreamerV3 codebase. The repository will include:

- A shared training and evaluation pipeline with a single configuration interface for selecting backbone, environment suite, and sub-experiment options (dynamic frequency-based sampling, CPC loss).
- Reproduction scripts and configuration files sufficient to replicate all reported results.
- A verified GRU baseline reproducing published DreamerV3 performance within expected variance on all three evaluation suites, serving as the reference point for all comparisons.

5.2 Experimental Results

Results from a minimum of 5 seeds per model–environment configuration, reported as:

- Mean and standard deviation of episodic return across seeds for each backbone on each evaluation suite (Atari100k, BSuite, POPGym).

- Interquartile mean (IQM) and performance profiles following the methodology recommended by Agarwal et al.[13], to ensure statistically robust comparison across tasks and seeds.
- Per-environment learning curves at fixed evaluation intervals to characterize convergence behavior, not only final performance.
- Wall-clock training time per environment step for each backbone, measured on identical hardware, to quantify the practical efficiency implications of parallelizable (SSM, Transformer) vs. sequential (GRU) training.

5.3 Ablation Analysis

Controlled ablation results isolating the contribution of each sub-experiment:

- Dynamic frequency-based sampling vs. uniform sampling for each backbone, to determine whether the benefit demonstrated by Drama[4] generalizes across architectures.
- CPC loss vs. standard loss for each backbone, with particular attention to whether Mamba-2 benefits from CPC to a degree comparable to the transformer, as hypothesized from the transformer–SSM duality[5]. We will report the magnitude of CPC's effect on each backbone individually to allow fine-grained comparison.

5.4 Paper

A detailed paper documenting methodology, experimental results, and analysis, including:

- A discussion of any implementation decisions required to reconcile architectural differences between backbones within the unified harness, and their potential impact on results.
- Failure mode analysis for any backbone that substantially underperforms, examining whether the cause is architectural (e.g., state compression in SSSMs) or incidental (e.g., hyperparameter sensitivity).

5.5 Presentation

A presentation summarizing findings, challenges encountered during implementation, and proposed future research directions informed by the results.

6 Dataset and Evaluation Metrics

6.1 Evaluation Suites

We evaluate across three environment suites, selected to test both general control performance and memory-specific capabilities. This mirrors the evaluation

strategy of Samsami et al.[3], who demonstrated that memory-intensive benchmarks most clearly expose the limitations of GRU-based world models.

6.1.1 Atari100k

The Atari100k benchmark[**simple**, 14] restricts agents to 100k environment interactions (400k frames with standard frame-skipping) across the standard set of 26 games introduced by Kaiser et al.[**simple**]. This benchmark is used by all five motivating backbone-swap papers, making it the natural common denominator for comparison. It primarily tests sample-efficient visual control rather than long-term memory, serving as our baseline for ensuring that backbone changes do not regress general performance. We use sticky actions ($p = 0.25$) following current best practices[15].

6.1.2 BSuite

The Behaviour Suite for Reinforcement Learning[16] provides targeted diagnostic experiments that isolate specific agent capabilities. We focus on the memory-relevant subset:

- **Memory Length**: a T-maze task parameterized by corridor length N, requiring the agent to recall a context bit presented at the start of the episode after N steps of null observation. We evaluate across the full sweep of lengths to characterize the memory horizon of each backbone.
- **Memory Size**: extends Memory Length by requiring recall of multiple context bits simultaneously, testing memory capacity rather than just duration.
- **Discounting Chain**: a chain MDP of configurable length where the agent must propagate value over long horizons, directly testing credit assignment.

Following the protocol of Hafner et al.[1], we run 10 seeds per BSuite configuration. BSuite environments use a single environment instance by design.

6.1.3 POPGym

The Partially Observable Process Gym[17] provides a diverse collection of 15 POMDP environments with multiple difficulty levels. Following the task selection of Samsami et al.[3] and Ni et al., we evaluate on the three most memory-intensive environments:

- **RepeatPrevious**: the agent must repeat an observation seen a fixed number of steps ago, directly testing sequential memorization.
- **Autoencode**: the agent must reproduce a sequence of observations after a delay, requiring multi-step memory encoding and retrieval.

- **Concentration**: a card-matching game requiring the agent to remember the locations and identities of previously revealed cards.

Each environment is evaluated at all three difficulty levels (Easy, Medium, Hard), where difficulty scales the number of actions or observations the agent must track simultaneously. Increased difficulty requires correspondingly longer effective memory horizons, providing a natural curriculum for distinguishing backbone capacity.

6.2 Evaluation Metrics

We adopt the evaluation methodology of Agarwal et al.[13] throughout, which has become the standard for statistically rigorous comparison in deep RL.

6.2.1 Score Normalization

For Atari100k, raw scores are normalized to the human baseline following the standard formula:

$$\text{HNS} = \frac{\text{agent score} - \text{random score}}{\text{human score} - \text{random score}}$$

using the human and random baselines from Bellemare et al.[14]. For BSuite, we use the native $[0, 1]$ scoring provided by the benchmark. For POPGym, we report raw episodic return, as human baselines are not available for these environments.

6.2.2 Aggregate Metrics

We report the following aggregate statistics across tasks within each suite:

- **Interquartile Mean (IQM)**: the mean of scores between the 25th and 75th percentiles across tasks and seeds. IQM is more robust to outlier tasks than the mean while being more statistically efficient than the median[13].
- **Optimality Gap**: the amount by which the agent falls short of a reference threshold (1.0 HNS for Atari100k), averaged across tasks. This highlights underperformance on difficult tasks that IQM may obscure.
- **Performance Profiles**: cumulative distribution functions of per-task scores across all runs, following Agarwal et al.[13]. These visualize the full distribution of performance and are more informative than any single scalar metric.

All aggregate metrics are reported with 95% stratified bootstrap confidence intervals computed over both tasks and seeds.

6.2.3 Per-Environment Reporting

In addition to aggregates, we report per-environment learning curves at fixed evaluation intervals (every 10k steps for Atari100k, every episode for BSuite). This serves two purposes: (1) characterizing convergence speed, which may differ substantially between backbones even when final performance is similar, and (2) identifying environments where a particular backbone catastrophically fails, which aggregate metrics can mask.

6.2.4 Computational Efficiency

We report wall-clock training time per environment step for each backbone, measured on identical hardware (single H100 GPU). This quantifies the practical benefit of parallelizable training (SSMs, Transformers) over the sequential GRU, which is one of the primary stated motivations for backbone replacement in the literature[3, 4].

6.2.5 Statistical Testing

For pairwise backbone comparisons, we compute the probability of improvement[13] — the probability that a randomly selected run-task pair from backbone A outperforms one from backbone B — as a nonparametric alternative to significance testing. We consider a difference meaningful when the probability of improvement exceeds 0.5 with non-overlapping 95% confidence intervals.

7 Timeline

To ensure consistent progress and a viable completion, the project will be carried out over a period of nine weeks and divided into the following phases:

7.1 Weeks 1–2: Foundation & Baselines:

Setting up the environment, getting acquainted with the DreamerV3 source, and reproducing the baseline GRU-based RSSM results on a chosen subset of environments.

7.2 Weeks 3: Architecture Integration:

Putting the single code base into practice and combining the standardized hidden state policy with the S3M, Mamba-2, S5, and Transformer backbones.

7.3 Weeks 4–5: Core Ablation Training:

Training and fine-tuning the implemented backbones in a sequential manner, monitoring the learning curves and confirming that the sequence models are operating appropriately within the task-agnostic constraints.

7.4 Weeks 6–7: Sub-Experiments:

Implementation of Dynamic Frequency-based Sampling and Contrastive Predictive Coding (CPC) loss. Starting training runs for all architectures in these sub-experiments.

7.5 Weeks 8: Evaluation & Analysis:

Aggregating results, generating learning curves, and examining the impact of each backbone on long-range credit assignment and overall sample efficiency.

7.6 Weeks 9: Deliverable Generation:

Finalizing the repository, drafting the final project report, and preparing the slide deck for the class presentation.

References

[1] Danijar Hafner et al. "Mastering Diverse Control Tasks through World Models". In: *Nature* (2025), pp. 647–653. DOI: 10.1038/s41586-025-08744-2. URL: https://doi.org/10.1038/s41586-025-08744-2.

[2] John Schulman et al. *Proximal Policy Optimization Algorithms*. 2017. arXiv: 1707.06347 [cs.LG]. URL: https://arxiv.org/abs/1707.06347.

[3] Mohammad Reza Samsami et al. "Mastering Memory Tasks with World Models". In: *The Twelfth International Conference on Learning Representations*. 2024. URL: https://openreview.net/forum?id=1vDArHJ68h.

[4] Wenlong Wang et al. "Drama: Mamba-Enabled Model-Based Reinforcement Learning Is Sample and Parameter Efficient". In: *The Thirteenth International Conference on Learning Representations*. 2025. URL: https://openreview.net/forum?id=7XIkRgYjK3.

[5] Tri Dao and Albert Gu. "Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality". In: *International Conference on Machine Learning (ICML)*. Vol. 235. 2024, pp. 10041–10071. URL: https://proceedings.mlr.press/v235/dao24a.html.

[6] Paul Mattes, Rainer Schlosser, and Ralf Herbrich. "Hieros: Hierarchical Imagination on Structured State Space Sequence World Models". In: *Proceedings of the 41st International Conference on Machine Learning*. 2024. URL: https://proceedings.mlr.press/v235/mattes24a.html.

[7] Jimmy T. H. Smith, Andrew Warrington, and Scott W. Linderman. "Simplified State Space Layers for Sequence Modeling". In: *The Eleventh International Conference on Learning Representations*. 2023. URL: https://openreview.net/forum?id=Ai8Hw3AXqks.

[8] Weipu Zhang et al. "STORM: Efficient Stochastic Transformer based World Models for Reinforcement Learning". In: *Thirty-seventh Conference on Neural Information Processing Systems*. 2023. URL: https://openreview.net/forum?id=WxnrX42rnS.

[9] Maxime Burchi and Radu Timofte. "Learning Transformer-based World Models with Contrastive Predictive Coding". In: *The Thirteenth International Conference on Learning Representations*. 2025. URL: https://openreview.net/forum?id=YK9G4Htdew.

[10] Ashish Vaswani et al. "Attention Is All You Need". In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017, pp. 5998–6008. URL: https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

[11] Zihang Dai et al. "Transformer-XL: Attentive Language Models beyond a Fixed-Length Context". In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2019, pp. 2978–2988. DOI: 10.18653/v1/P19-1285. URL: https://doi.org/10.18653/v1/P19-1285.

[12] Jan Robine et al. "Transformer-based World Models Are Happy With 100k Interactions". In: *The Eleventh International Conference on Learning Representations*. 2023. URL: https://openreview.net/forum?id=TdBaDGCpjly.

[13] Rishabh Agarwal et al. "Deep Reinforcement Learning at the Edge of the Statistical Precipice". In: *Advances in Neural Information Processing Systems* 34 (2021). URL: https://proceedings.neurips.cc/paper/2021/hash/f514cec81cb148559cf475e7426eed5e-Abstract.html.

[14] M. G. Bellemare et al. "The Arcade Learning Environment: An Evaluation Platform for General Agents". In: *Journal of Artificial Intelligence Research* 47 (June 2013), pp. 253–279. URL: https://dl.acm.org/doi/abs/10.5555/2566972.2566979.

[15] Marlos C. Machado et al. "Revisiting the Arcade Learning Environment: Evaluation Protocols and Open Problems for General Agents". In: *Journal of Artificial Intelligence Research* 61 (2018), pp. 523–562. DOI: 10.1613/jair.5699. URL: https://doi.org/10.1613/jair.5699.

[16] Ian Osband et al. "Behaviour Suite for Reinforcement Learning". In: *International Conference on Learning Representations*. 2020. URL: https://openreview.net/forum?id=rygf-kSYwH.

[17] Steven Morad et al. "POPGym: Benchmarking Partially Observable Reinforcement Learning". In: *The Eleventh International Conference on Learning Representations*. 2023. URL: https://openreview.net/forum?id=chDrutUTs0K.